Deep Learning

UW ECE 657A - Core Topic

Mark Crowley

Lecture Outline

Lecture Outline

Linear Regression vs. Logistic Regression

- A simple type of **Generalized Linear Model**
- Linear regression learns a function to predict a continuous variable output of continuous or discrete input variables

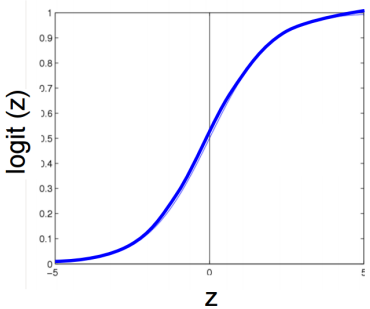
$$= b_0 + \sum(b_i X_i) + \epsilon$$

- Logistic regression predicts the probability of an outcome, the appropriate class for an input vector or the **odds** of one outcome being more likely than another.

Logistic Regression

Define probability of label being 1 by fitting a linear weight vector to the logistic (a.k.a sigmoid) function.

$$P(Y=1|X) = \frac{1}{1+e^{-(w_0+\sum_i w_i x_i)}} = \frac{1}{1+\exp(-(w_0+\sum_i w_i x_i))}$$

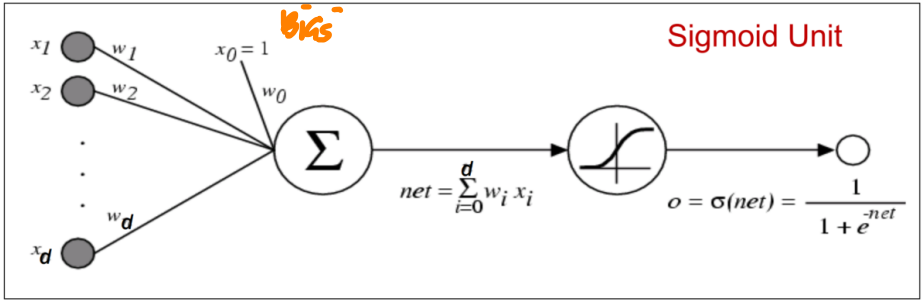


Advantage of this is you can turn the continuous $[-\infty, \infty]$ feature information into $[0, 1]$ and treat it like a probability.

bias: w_0 is called the bias, it basically adjusts the sigmoid curve to the left or right, biasing what the expected outcome is. The other weights w_i adjust the steepness of the curve in each

Logistic Regression as a Graphical Model

$$o(\mathrm{x}) = \sigma(w^T x_i) = \sigma(w_0 + \sum_i w_i x_i) = \frac{1}{1 + \exp(-(w_0 + \sum_i w_i x_i))}$$



Properties of Logistic Regression

- Very simple yet powerful classification method
- Relatively easy to fit to data using gradient descent, many methods for doing this
- Learned parameters can be interpreted as computing the **log odds**, the logarithm of the ratio of successes to failures

$$\begin{aligned} LO &\sim \log \frac{p(Y=1|X)}{p(Y=0|X)} \\ &= w^T \mathbf{x} \end{aligned}$$

- If x_0 : number of cigarettes per day
- x_1 : minutes of excercise
- $y = 1$: getting lung cancer
- Then if parameters learned at $\hat{w} = (1.3, -1.1)$, it means each cigarette raises odds of cancer by factor of $e^{1.3}$

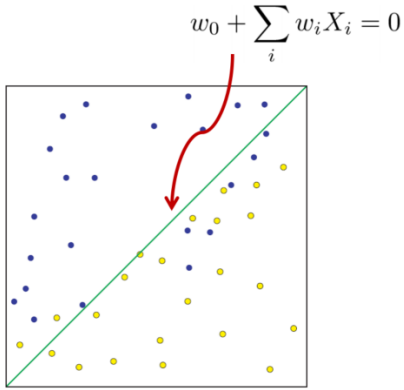
Logistic Regression Used as a Classifier

Logistic Regression can be used as a simple linear classifier.

- Compare probabilities of each class $P(Y = 0|X)$ and $P(Y = 1|X)$.
- Treat the halfway point on the sigmoid as the decision boundary.

$P(Y = 1|X) > 0.5$ classify X in class 1

$$w_0 + \sum_i w_i x_i = 0$$



Training Logistic Regression Model via Gradient Descent

- Can't easily perform Maximum Likelihood Estimation
- The negative log-likehood of the logistic function is given by NLL and it's gradient by g

$$NLL(w) = \sum_{i=1}^{N} \log\left(1 + \exp(-(w_0 + \sum_i w_i x_i))\right)$$

$$g = \frac{\partial}{\partial w} = \sum_i (\sigma(w^T x_i) - y_i) x_i$$

Then we can update the parameters iteratively

$$\theta_{k+1} = \theta_k - \eta_k g_k$$

where η_k is the learning rate or step size.

Other Linear Classifiers

- k Nearest Neighbours - label data points based on similarity to others using distance metric over defined neighbourhood.
- Naive Bayes - probabilistic model that assumes independence of input features. Uses Bayes rule to update a distribution over the inputs to minimizes entropy of the outputs. Classification by threshold on probability.
- Support Vector Machines - learns a discriminant function and threshold to divide points into two classes.

More Powerful ML Classifiers

Naive Bayes is a probabilistic model that assumes independence of input features and uses Bayes rule to update a distribution over the inputs that minimizes entropy of the outputs. Classification is performed by choosing the more likely class or apply a threshold τ to determine if a class is valid.

Support Vector Machines (SVMs)

- SVM learns a discriminant function and threshold to divide points into two classes.
- Originally this was a line $\mathrm{w}^T\mathrm{x} + b$ where weights w and offset b are learned so that possitive training points are above the line.
- More generally we can learn a nonlinear function using the **kernal trick**.

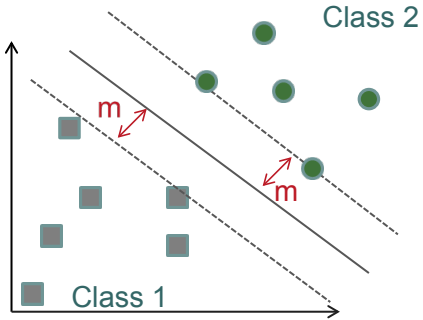
We rewrite the linear system as a dot product:

$$\mathrm{w}^T\mathrm{x} + b = b + \sum_{i=1}^{m} \alpha_i \mathrm{x}^T\mathrm{x}^{(i)}$$

where

- $\mathrm{x}^{(i)}$ is a training example
- α is a vector of coefficients

Support Vector Machines



Support Vector Machines (SVMs)

- More generally we can learn a nonlinear function using the **kernal trick**.

We rewrite the linear system as a dot product:

$$\mathrm{w}^T\mathrm{x} + b = b + \sum_{i=1}^{m} \alpha_i \mathrm{x}^T \mathrm{x}^{(i)}$$

$$f(x) = b + \sum_{i} \alpha_i k(\mathrm{x}, \mathrm{x}^{(i)})$$

where

- $\mathrm{x}^{(i)}$ is a training example
- α is a vector of coefficients
- $k(\mathrm{x}, \mathrm{x}^{(i)}) = \phi(\mathrm{x}) \cdot \phi(\mathrm{x})^T$ is a kernal.

Pros and Cons of Kernal Methods

Pro:

- They map low dimensional linear space to higher dimensional nonlinear space.

Cons:

- Evaluation is expensive - linear in the weighted data points usd $\alpha_i k(\mathrm{xx}^{(i)})$
  - kernal SVMs avoid this problem because most of $\alpha_i = 0$, only the **support vectors** $\mathrm{x}^{(i)}$ are computed.
- Training is expensive - for large datasets
- Do not Generalize well - the kernal determines what kind of general patterns it can learn.
- Choosing the kernal is hard - too general or too specific to the problem, how to choose it?

Lecture Outline

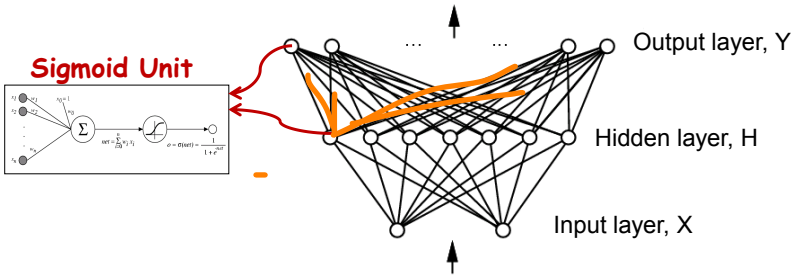
Neural Networks

- A Neural Network (NN) connects many nonlinear (classically logistic/sigmoid) units together into a network
- Central difference from the Perceptron: a layer of **hidden units**
- Also called: Multi-layer Perceptrons (MLP), Artificial Neural Networks (ANNs)

Neural Networks to learn $f : X \rightarrow Y$

- f can be a non-linear function
- X (vector of) continuous and/or discrete variables
- Y (vector of) continuous and/or discrete variables

Feedforward Neural networks - Represent f by network of non-linear (logistic/sigmoid/ReLU) units:



Basic Three Layer Neural Network

Input Layer

- vector data, each input collects **one feature**/dimension of the data and passes it on to the (first) hidden layer.

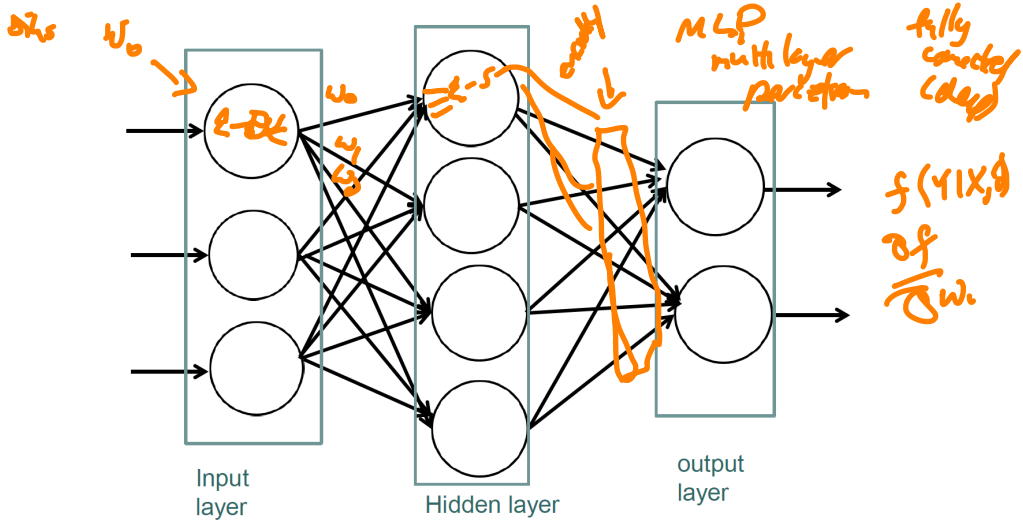
Hidden Layer

- Each hidden unit computes a weighted sum of all the units from the input layer (or any previous layer) and passes it through a **nonlinear activation function**.

Output Layer

- Each output unit computes a weighted sum of all the hidden units and passes it through a (possibly nonlinear) **threshold function**.

Three-layer Artificial Neural Network (ANN)



Properties of Neural Networks

- Given a large enough layer of hidden units (or multiple layers) a NN can represent **any function**. → Universal Approximators
- One challenge is regularization: how many hidden units or layers to include in the network? Number of inputs and outputs are provided but internal structure can be arbitrary.
  - too few units, network will have too few parameters, may not be able to learn complex functions
  - too many units, network will be overparameterized and won't be forced to learn a generalizable model

Representation Learning: classic statistical machine learning is about learning functions to map input data to output. But Neural Networks, and especially Deep Learning, are more about learning **a representation** in order to perform classification or some other task.

Lecture Outline

A Short History

40's Early work in NN goes back to the 40's with a simple model of the neuron by McCulloh and Pitt as a summing and thresholding devices.

A Short History

40's Early work in NN goes back to the 40's with a simple model of the neuron by McCulloh and Pitt as a summing and thresholding devices.

1958 Rosenblatt in 1958 introduced the **Perceptron**,a two layer network (one input layer and one output node with a bias in addition to the input features.

1969 Marvin Minsky: 1969. Perceptrons are 'just' linear, AI goes logical, beginning of "AI Winter"

A Short History

40's Early work in NN goes back to the 40's with a simple model of the neuron by McCulloh and Pitt as a summing and thresholding devices.

1958 Rosenblatt in 1958 introduced the **Perceptron**,a two layer network (one input layer and one output node with a bias in addition to the input features.

1969 Marvin Minsky: 1969. Perceptrons are 'just' linear, AI goes logical, beginning of "AI Winter"

1980s Neural Network resurgence: Backpropagation (updating weights by gradient descent)

A Short History

- 40's Early work in NN goes back to the 40's with a simple model of the neuron by McCulloh and Pitt as a summing and thresholding devices.
- 1958 Rosenblatt in 1958 introduced the **Perceptron**,a two layer network (one input layer and one output node with a bias in addition to the input features.
- 1969 Marvin Minsky: 1969. Perceptrons are 'just' linear, AI goes logical, beginning of "AI Winter"
- 1980s Neural Network resurgence: Backpropagation (updating weights by gradient descent)
- 1990s SVMs! Kernals can do **anything!** (no, they can't)

A Short History

1993 `LeNet 1` for digit recognition

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

2006 Neural Network outperform RBF SVM on MNIST handwriting dataset (Hinton et al.)

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

2006 Neural Network outperform RBF SVM on MNIST handwriting dataset (Hinton et al.)

2012 `AlexNet` for **ImageNet** challenge - this algorithm beat competition by error rate of 16% vs 26% for next best

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

2006 Neural Network outperform RBF SVM on MNIST handwriting dataset (Hinton et al.)

2012 `AlexNet` for **ImageNet** challenge - this algorithm beat competition by error rate of 16% vs 26% for next best

- ImageNet : contains 15 million annotated images in over 22,000 categories.

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

2006 Neural Network outperform RBF SVM on MNIST handwriting dataset (Hinton et al.)

2012 `AlexNet` for **ImageNet** challenge - this algorithm beat competition by error rate of 16% vs 26% for next best

- ImageNet : contains 15 million annotated images in over 22,000 categories.
- ZFNet paper (2013) extends this and has good description of network structure

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

2006 Neural Network outperform RBF SVM on MNIST handwriting dataset (Hinton et al.)

2012 `AlexNet` for **ImageNet** challenge - this algorithm beat competition by error rate of 16% vs 26% for next best

- ImageNet : contains 15 million annotated images in over 22,000 categories.
- ZFNet paper (2013) extends this and has good description of network structure

2014 `GoogLeNet` added many layers and introduced **inception** modules (allows parallel computation rather than serially connected CNN layers)

A Short History

1993 `LeNet 1` for digit recognition

2003 Deep Learning (Convolutional Nets Dropout/RBMs, Deep Belief Networks)

1986, 2006 Restricted Boltzman Machines

2006 Neural Network outperform RBF SVM on MNIST handwriting dataset (Hinton et al.)

2012 `AlexNet` for **ImageNet** challenge - this algorithm beat competition by error rate of 16% vs 26% for next best

- ImageNet : contains 15 million annotated images in over 22,000 categories.
- ZFNet paper (2013) extends this and has good description of network structure

2014 `GoogLeNet` added many layers and introduced **inception** modules (allows parallel computation rather than serially connected CNN layers)

2012-present Google Cat Youtube, speech recognition, self driving cars, computer defeats world Go champion, ...

A "New" Age of Neural Networks

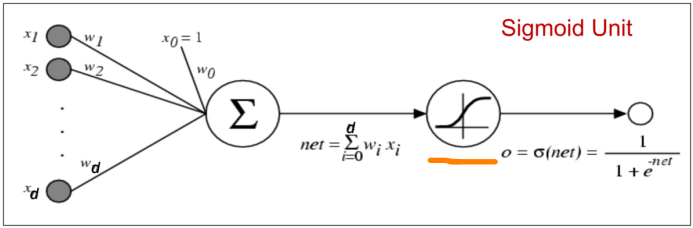
- Geoffrey Hinton (UofT, now Google Research), paper references
  - early work on back-propogation
  - Restricted Boltzman Machines - first deep architecture, idea for unsupervised pretraining of layers for RBMs was used for first successful use of deep networks.
  - Several applications of Deep Learning to image recognition, speech recognition,...
  - Regularization methods: dropout, data augmentation
- Yoshua Bengio (University of Montreal, MILA, ElememntAI, ...)
  - ReLU improves efficiency
  - unsupervised representation learning of autoencoders
  - word2vec - learn general representation of words for classification, translation, etc.
  - General Adversarial Networks - an exciting new approach to training
- Yann LeCun (NYU, now Facebook AI Research)
  - Convolutional Neural Networks

Lecture Outline

Properties of Neural Networks

- The hidden layer will have a number of units (design parameter).
- The hidden layers allow extracting more complex features and facilitates solving complex problems.
- Connection between the units (neurons) of all the layers can be **forward**, backward or both.
- Units can be fully or partially connected.
- Different arrangements make different types or models of the NN.
- Each unit will have a **activation function** (a thresholding function) and connections between the units that have weights

Using Sigmoid Unit as an Activation Function



$$o(\mathrm{x}) = \sigma(w^T x_i) = \sigma(w_0 + \sum_i w_i x_i) = \frac{1}{1 + \exp(-(w_0 + \sum_i w_i x_i))}$$

Differentiable: $\frac{\mathrm{d}\sigma(x)}{\mathrm{d}x} = \sigma(x)(1 - \sigma(x))$

Net Activation

$$net_j = \sum_{i=1}^{d} x_i w_{ji} + w_{j0} = \sum_{i=0}^{d} x_i w_{ji}$$

where:

- i indexes the inputs units
- j indexes the hidden units
- W_{ji} denotes the weights on input to hidden layer ("synaptic weights")

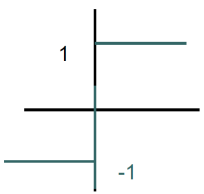
Hidden Layer: Adding Nonlinearity

- Each hidden unit emits an output that is a nonlinear **activation function** of its net activiation.

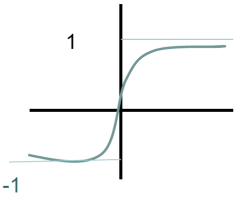
$$y_j = f(net_j)$$

- This is essential to neural networks power, if it's linear then it all becomes just linear regression.
- The output is thus thresholded through this nonlinear activation function.

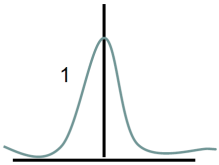
Activation Functions



a) Threshold



b) Sigmoid

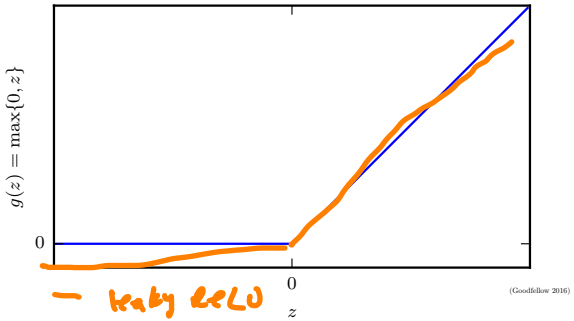


c) Gaussian

$$F_T(x) = \begin{cases} 1 & if\ x > \tau \\ -1 & otherwise \end{cases} \qquad F_S(x) = \frac{1}{\left(1 + e^{-cx}\right)} \qquad F_G(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$$

- `tanh` was another common function.
- `sigmoid` is now discourage except for final layer to obtain probabilities. Can **over-saturate** easily.
- ReLU is the new standard activation function to use.

Rectified Linear Activation



- **Rectified Linear Units (ReLU)** have become standard $\max(0, net_j)$
  - strong signals are alwasy easy to distinguish
  - most values are zero, deritive is mostly zero
  - they do not saturate as easily as sigmoid
- new Exponential linear units - evidence that they perform better than ReLU in some

Output Layer

Regardless of activation function, the output layer then combines all the previous hidden layers.

$$net_k = \sum_{j=1}^{n_H} y_j w_{kj} + w_{k0}$$

where:

- k indexes the output units
- n_H denotes the number of hidden units in the previous layer

Output Layer

The output layer adds *another* level of thresholding, usually nonlinear as well.

$$z_k = f(net_k)$$

$f(net_k)$ could be any thresholding function such as the previous activation functions.

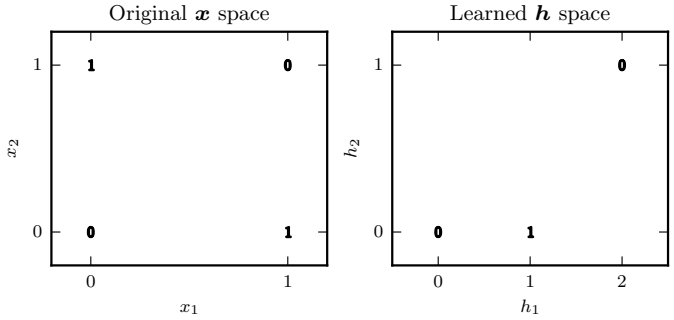
Lecture Outline

Outline

Failing on the XOR Problem

How is XOR a hard problem? It is if you're too linear. XOR is the simplest **nonlinear** classification problem.

- Two binary features x_1 and x_2 and four patterns to be assigned the correct labels `True` or `False`



Neural Nets Can Model XOR Problem

Use a simple `sign` activation function for hidden and output units:

$$f(net) = 1 \text{ if } net \geq 0$$
$$= -1 \text{ if } net < 0$$

- First hidden unit:
  - y_1 computes $x_1 + x_2 + 0.5 = 0$
  - $y_1 = 1$ if $net_1/ge0$, $y_1 = -1$ otherwise
- Second hidden unit:
  - y_1 computes $x_1 + x_2 - 1.5 = 0$
  - $y_2 = 1$ if $net_2/ge0$, $y_2 = -1$ otherwise
- single output unit z_1:
  - emits $z_1 = 1$ iff $y_1 = 1$ and $y_2 = 1$.
  - y_1 models AND gate
  - y_2 models OR gate
  - z_1 models y_1 AND NOT y_2

Full Neural Network Descriminant Function

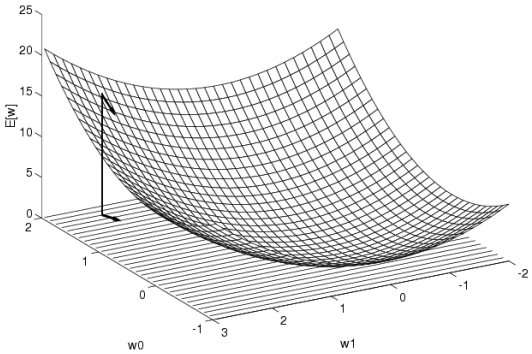
The class of decision functions that can be described by a three-layer neural network are:

$$g_k(\mathbf{x}) = z_k = f\left(\sum_{j=1}^{n_H} w_{kj} f\left(\sum_{i=1}^{d} w_{ji} x_i + w_{j0}\right) + w_0\right)$$

- The **Universal Approximation Theorem** [Hornik, 1989] shows that this covers any possible decision function mapping continuous inputs to a finite set of classes to any desired level of accuracy.
- But it does not say how many hidden unites would be needed.
- In general, for a single layer it could be exponential (degrees of freedom).
- But, using more layers can reduce the number of hidden units needed and make it more generalizable.

Lecture Outline

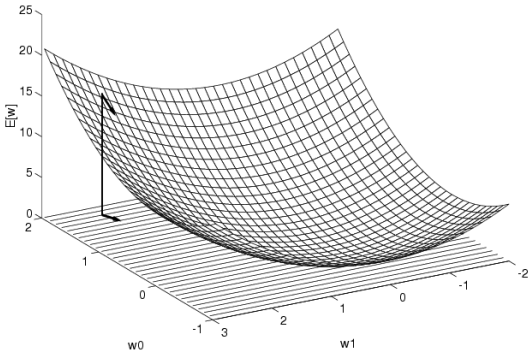
Gradient Descent



Error Function: Mean Squared Error,

cross-entropy loss, etc.

Gradient Descent

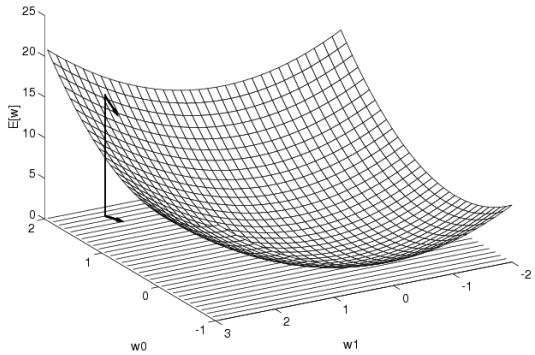


Error Function: Mean Squared Error,

cross-entropy loss, etc.

Gradient: $\bigtriangledown E[\mathrm{w}] = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_d}\right]$

Gradient Descent



Error Function: Mean Squared Error,

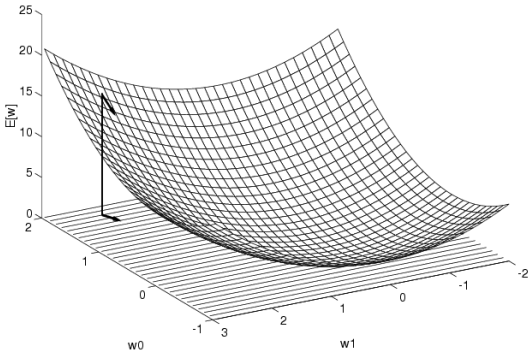
cross-entropy loss, etc.

Gradient: $\bigtriangledown E[\mathrm{w}] = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_d}\right]$

Training Update Rule: $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$ where η is the training rate.

Note: For regression, others, this gradient is convex. In NNs it is not. So we must solve

Gradient Descent



Error Function: Mean Squared Error,

cross-entropy loss, etc.

Gradient: $\bigtriangledown E[\mathrm{w}] = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_d}\right]$

Training Update Rule: $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$ where η is the training rate.

Note: For regression, others, this gradient is convex. In NNs it is not. So we must solve

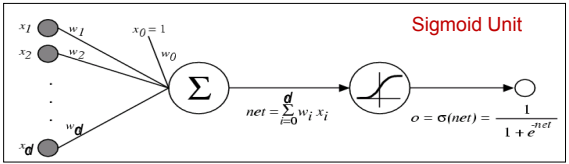
Incremental Gradient Descent

Let error function be : $E_l[\mathrm{w}] = \frac{1}{2}(y^l - o^l)^2$
Do until satisfied:

- For each training example l in D
  1. Compute the gradient $\bigtriangledown E[\mathrm{w}]$
  2. update weights : $\mathrm{w} = \mathrm{w} - \eta \bigtriangledown E[\mathrm{w}]$

Note: can also use batch gradient descent on many points at once.

Error Gradient for a Sigmoid Unit



$$
\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{l \in D} (y^l - o^l)^2 \\
&= \frac{1}{2} \sum_{l} \frac{\partial}{\partial w_i} (y^l - o^l)^2 \\
&= \frac{1}{2} \sum_{l} 2(y^l - o^l) \frac{\partial}{\partial w_i} (y^l - o^l) \\
&= \sum_{l} (y^l - o^l) \left(-\frac{\partial o^l}{\partial w_i} \right) \\
&= -\sum_{l} (y^l - o^l) \frac{\partial o^l}{\partial net^l} \frac{\partial net^l}{\partial w_i}
\end{aligned}
$$

But we know:

$$\frac{\partial o^l}{\partial net^l} = \frac{\partial \sigma(net^l)}{\partial net^l} = o^l(1 - o^l)$$

$$\frac{\partial net^l}{\partial w_i} = \frac{\partial (\vec{w} \cdot \vec{x}^l)}{\partial w_i} = x_i^l$$

So:

$$\frac{\partial E}{\partial w_i} = -\sum_{l \in D} (y^l - o^l) o^l (1 - o^l) x_i^l$$

(Slides from Tom Mitchell ML Course, CMU, 2010)

Backpropagation Algorithm

We need an iterative algorithm for getting the gradient efficiently.
For each training example:

1. Forward propagation: Input the training example to the network and compute outputs
2. Compute output units errors:

$$\delta_k^l = o_k^l(1 - o_k^l)(y_k^l - o_k^l)$$

3. Compute hidden units errors:

$$\delta_h^l = o_h^l(1 - o_h^l) \sum_k w_{h,k} \delta_k^l$$

4. Update network weights:

$$w_{i,j} = w_{i,j} + \Delta w_{i,j}^l = w_{i,j} + \eta \delta_j^l o_i^l$$

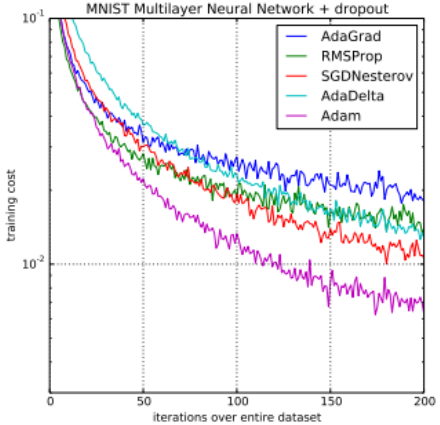
More about Backpropagation

- Still underlies all of Deep Learning
- Can be easily peformed on mulidimensional outputs.
- No guarantee of global optimal solution, only local, but often good enough.
- Can be paralellized very well, most deep learning tools now do this on GPUs automatically.
- Remember that it is minimizing errors on *training* examples, ability to generalize relies on network structure, training approach, other factors.

Other Optimizers

- **SGD (Stochastic Gradient Descent):** Can perform best but various additions need to be made and learning rate need to be correctly tuned.
- **Adam (adapative moment estimate):** For relatively large datasets Adam is very robust. Converges quickly and gives pretty good performance.
  - keeps track of gradient learning rates and momentum *per dimension*
  - uses variance (2nd moment) rather than means and scales steps based on recent magnituges
- **L-BFGS :** Converges faster and with better solutions on small datasets. Very memory efficient, scales linearly with variables. Estimates the inverse Hessian matrix (2nd order derivatives)
- See a good overview of the best methods here: [Rud17] (`https://arxiv.org/abs/1609.04747`)

Adam Optimizer



Lecture Outline

References


Sebastian Ruder, *An overview of gradient descent optimization algorithms*, arXiv:1609.04747 [cs] (2017), arXiv: 1609.04747.